

ПРИМЕНЕНИЕ МЕХАНИЗМА МНОГОУРОВНЕВОЙ СПРАВЕДЛИВОЙ ОЧЕРЕДИ ДЛЯ СНИЖЕНИЯ УЩЕРБА ОТ АТАК ОТКАЗА В ОБСЛУЖИВАНИИ В ПРОГРАММНО-КОНФИГУРИРУЕМЫХ СЕТЯХ

М.Ю. Рыгов, Р.Ю. Калашников, А.А. Горелов

Концепция программно-конфигурируемых сетей (SDN) стремительно набирает популярность в управлении сетевой инфраструктурой центров обработки данных и операторов связи. К её ключевым функциям относятся мониторинг, детальное управление, гибкость и масштабируемость. Но вместе с тем, централизованное управление SDN делает его уязвимым для различных типов атак, таких как спуфинг и отказ в обслуживании (DoS). DoS-атаки оказывают наиболее серьезное воздействие, поскольку они снижают производительность сети из-за перегрузки ее различных компонентов, то есть контроллера, коммутатора и канала управления. Существующие подходы справляются с DoS-атаками в SDN либо путем отбрасывания вредоносных пакетов, либо путем объединения правил потока, что приводит к потерям легитимного трафика. Для уменьшения последствий DoS-атак в этой статье предлагается использование механизма многоуровневой справедливой очереди, который обеспечивает совместное использование ресурсов контроллера с несколькими уровнями очередей, которые могут динамически расширяться и агрегироваться в зависимости от загруженности сети. Предлагаемый подход оценивается путем сравнения его с базовым контроллером SDN. Результаты моделирования показывают, что предлагаемый подход увеличивает производительность SDN с точки зрения использования пропускной способности канала управления.

Ключевые слова: сетевая безопасность, отказ в обслуживании, программно-конфигурируемые сети.

Введение

Программно-конфигурируемые сети (SDN) становятся все более популярной моделью сетевой инфраструктуры благодаря своей простой конфигурации и легкому управлению. SDN упрощает мониторинг передаваемых данных и обеспечивает централизованный контроль над сетевым трафиком. Таким образом, программно-конфигурируемые сети могут управлять всем сетевым трафиком с помощью различных протоколов, таких как IPv4, IPv6, Интернет-протокол управляющих сообщений (ICMP), протокол управления передачей (TCP) и протокол пользовательских дейтаграмм (UDP) и т.д. Кроме того, SDN более масштабируемы и гибки по сравнению с традиционными сетями. В программно-конфигурируемых сетях уровень пересылки данных (коммутаторы, маршрутизаторы и т.д.) отделен от уровня управления (контроллера или операционной системы), что делает их оптимальной моделью инфраструктуры для крупномасштабных и высокоскоростных сетей. Уровень

управления посредством операционной системы координируется с уровнем приложения для выполнения различных задач, таких как обнаружение вторжений, балансировка нагрузки, маршрутизация и т.д. [1]. Как правило уровень управления использует для связи с уровнем пересылки данных протокол OpenFlow, который передает сообщения (Packet-In, Packet-Out, Flow-Add и т.д.) между коммутаторами и контроллером для обеспечения маршрутизации и управления сетью.

В OpenFlow каждый коммутатор хранит таблицу потоков с записями, которые добавляет контроллер для пересылки пакетов. Когда пакет прибывает в коммутатор, он сопоставляет заголовок пакета с записями потока в таблице, чтобы предпринять соответствующие действия. Если соответствующей записи нет, то заголовок пакета инкапсулируется в сообщение Packet-In и направляется контроллеру для выполнения подходящего действия. Контроллер добавляет запись потока в коммутатор, отправляя сообщение

Flow-Add, содержащее соответствующее действие. Остальные пакеты с таким же заголовком перемещаются в соответствии с инструкциями в соответствующем правиле потока. Каждому правилу назначается время жизни, которое определяет, как долго правило может существовать. При получении правила, коммутатор проверяет, заполнена ли его таблица потоков. Если нет, правило вставляется в таблицу потоков; в противном случае правило отбрасывается, и на контроллер будет отправлено сообщение об ошибке. На сегодняшний день протокол OpenFlow поддерживается сетевыми коммутаторами крупных производителей, такими как Juniper, HP, Cisco и др. [2].

Архитектура SDN предоставляет контроллеру широкий спектр возможностей по управлению уровнем данных и упрощает его конфигурацию. Кроме того, архитектура SDN по возможности использует общие модели и механизмы, что минимизирует требования к её стандартизации и интеграции. Это подразумевает использование существующих стандартов и общепринятых передовых практик.

В то же время безопасность программно-конфигурируемых сетей является важной проблемой, поскольку она не является частью архитектуры. SDN уязвимы для различных видов угроз безопасности, таких как спуфинг, подмена сообщений, отказ от авторства, раскрытие информации, отказ в обслуживании (DoS) и повышение привилегий. Среди этих угроз атаки отказа в обслуживании имеют наиболее разрушительный эффект, поскольку они снижают производительность всей сети за счет увеличения задержки и отбрасывания легитимных пакетов [3]. Злоумышленник может легко запустить DoS-атаку, массово отправляя пакеты с поддельными адресами источника. Это приведет к тому, что коммутатор отправит каждый пакет контроллеру в форме сообщения Packet-In. Эта атака приведет к перегрузке контроллера, канала управления (связи между контроллером и коммутатором) и таблицы потоков.

Злоумышленник может организовать DoS-атаку или ее распределенную версию (DDoS-атаку), чтобы перегрузить всю сеть,

исчерпав ресурсы контроллера (вычислительную мощность и память), или сегмент сети, исчерпав ресурсы коммутатора (буфер пакетов или таблицу потоков) [3].

Назначение правил коммутации в программно-конфигурируемых сетях возможно в двух режимах: проактивном и реактивном. Для проактивного режима контроллер сначала разбивает сетевые политики на правила потоков и устанавливает их на коммутаторах при загрузке контроллера. Для реактивного режима контроллер просчитывает и устанавливает правила только тогда, когда коммутатор явно запрашивает их. Реактивный режим позволяет коммутаторам быстро адаптироваться к изменениям топологии сети и не требует установки больших таблиц потоков в коммутаторах. [4]

Однако реактивная установка правил коммутации также делает контроллеры и коммутаторы SDN уязвимыми для угроз отказа в обслуживании. В частности, злоумышленник может вызвать переполнение таблиц потоков порождая большое количество запросов к хостам с адресами, неизвестными коммутатору. Последствия такой атаки включают в себя следующие последствия:

1. Перегрузка программного обеспечения коммутатора

Коммутатора-жертвы должен сгенерировать запрос потока и отправить его контроллеру. Поскольку коммутаторы обычно работают на процессорах относительно низкого уровня, они могут генерировать только ограниченное количество запросов в единицу времени. Таким образом, коммутаторы SDN являются узким местом и могут быть перегружены, в результате чего потоки трафика от легитимных узлов сети могут задерживаться или отбрасываться. Стоит отметить, что эта угроза имеет локальное влияние: ей подвержены только те хосты, которые напрямую подключены к коммутатору-жертве.

2. Перегрузка канала управления

Хотя коммутаторы поддерживают свои каналы управления с контроллером

независимо друг от друга, эти логически отдельные каналы могут иметь общие физические подключения. Таким образом, запросы потока, переданные коммутатором-жертвой, могут перегрузить общие узкие места, и обычные запросы потока, использующие эти ссылки, испытывают перегрузку. Кроме того, если буфер пакетов коммутатора-жертвы заполняется, коммутатор отправляет на контроллер целые пакеты (а не только заголовки), что приводит к еще большей загрузке полосы пропускания. По сравнению с угрозой перегрузки программного обеспечения коммутатора, которая затрагивает только те узлы, которые подключены к коммутатору-жертве, перегрузка канала управления влияет на все узлы с запросами потока, проходящими через перегруженные ссылки.

3. Истощение ресурсов контроллера

Когда запросы потока поступают на контроллер, для вычисления и установки правил используются ресурсы контроллера (ЦП, память, пропускная способность и т.д.). В случае, когда не предусмотрено защиты от лавинообразного потока запросов, ресурсы контроллера могут быть истощены, что приведет к отбрасыванию легитимных запросов. Подобно перегрузке канала управления, истощение ресурсов контроллера имеет последствия для всей сети, так как будут затронуты все коммутаторы, подключенные к контроллеру.

4. Переполнение таблицы потоков

Важной особенностью программно-конфигурируемых сетей является управление на основе потоков, и эта функция требует большого количества совпадающих полей, чем IP-сети. Например, последний стандарт OpenFlow определяет 45 совпадающих полей, 14 из которых являются обязательными [5]. Поскольку память коммутатора обычно является ограниченным ресурсом, типичные коммерческие коммутаторы OpenFlow поддерживают небольшое количество правил потока. Таким образом, учитывая, что контроллер успешно обработал множество поддельных запросов от злоумышленника и установил правила, таблица потоков коммутатора-жертвы быстро переполняется.

В результате чего отбрасываются правила для легитимных потоков. Подобно перегрузке программного обеспечения коммутатора, эта угроза также имеет локальное воздействие, поскольку она снижает пропускную способность только коммутаторов-жертв. Истощение ресурсов контроллера является наиболее опасным последствием DoS-атак в SDN, так как в этом случае снижается пропускная способность всей сети, а не отдельных сегментов [3].

Одной из эффективных мер противодействия подобной атаке является обеспечение справедливого распределения ресурсов контроллера между коммутаторами и хостами в сети. Проблема в том, что при наличии большого количества коммутаторов и хостов необходимо поддерживать большое количество очередей. Чтобы решить эту проблему, предлагается использовать механизм многоуровневой справедливой очереди, который позволяет динамически расширять и агрегировать очереди. Используя такой подход, контроллеру обычно требуется поддерживать лишь небольшое количество очередей, а когда происходят атаки, контроллер разбивает существующие очереди в несколько очередей более низкого уровня для изоляции вредоносных запросов.

Описание предложенного алгоритма

Одноуровневая справедливая очередь [6]

При использовании одноуровневой очереди контроллер поддерживает очередь для каждого коммутатора. Когда приходит запрос, контроллер ставит его в очередь соответствующего коммутатора. Затем контроллер использует взвешенную циклическую проверку для обработки запросов из этих очередей. Допустим, есть несколько коммутаторов $S_1, S_2, \dots, S_n$ с весами $w_1, w_2, \dots, w_n$ соответственно. Пусть W_{max} и W_{div} будут максимальным и наибольшим общим делителями весов соответственно.

Контроллер имеет параметр W , с начальным значением W_{max} . Контроллер в цикле перебирает все очереди коммутаторов и берет один пакет из очереди S_i , если $w_i \geq W$. На каждом шаге цикла W уменьшается на W_{div} . Если W становится меньше 0, то он снова

устанавливается в значение W_{max} . Так, например, предположим, что есть три коммутатора A , B и C с весами 3, 2 и 1 соответственно. Тогда контроллер будет опрашивать очереди в порядке $ABCA$.

Несмотря на то, что такой механизм обеспечивает справедливую обработку запросов среди коммутаторов, у него есть ограничения. Так в больших телекоммуникационных сетях поддержание очереди для каждого коммутатора может повлечь большие накладные расходы. Кроме того, хосты, находящиеся в том же сегменте сети, что и коммутатор, находящийся под атакой, могут быть изолированы, поскольку их запросы потока совместно используют одну и ту же очередь.

Многоуровневая справедливая очередь

Для решения вышеуказанных проблем предложено использовать механизм многоуровневой справедливой очереди [7] (рис. 1). Основная идея состоит в том, чтобы поддерживать лишь небольшое количество очередей на контроллере, когда нет атак, и динамически разбивать очередь на несколько под-очередей, когда ее размер превышает пороговое значение (что указывает на наличие в этой очереди злонамеренных запросов). В случае, когда размеры этих расширенных под-очередей падают ниже другого порога, они снова объединяются в одну очередь (рис. 2).

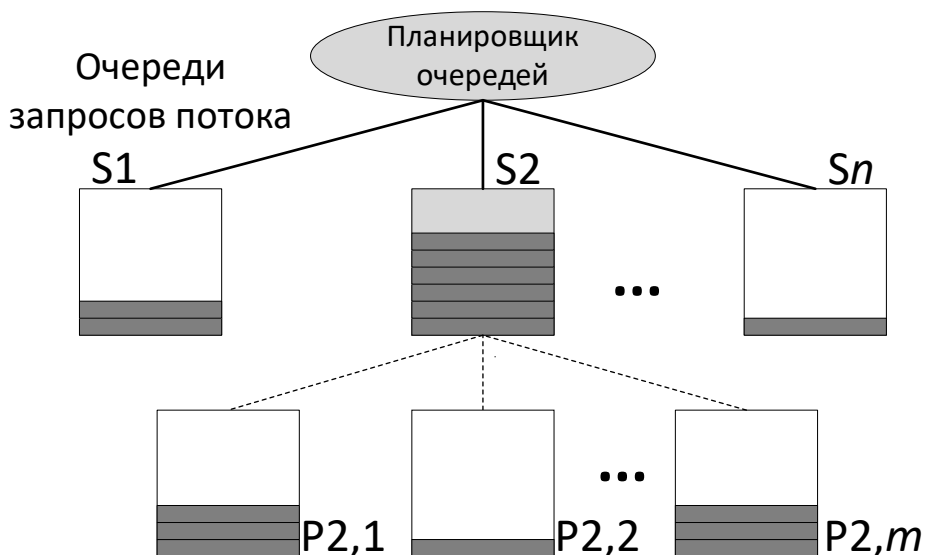


Рис. 1. Механизм работы многоуровневой справедливой очереди

Например, изначально каждая очередь соответствует группе коммутаторов. Когда размер очереди превышает пороговое значение, она разделяется на несколько очередей – по одной для каждого коммутатора. Если размер очереди на коммутатор все еще превышает пороговое

значение, она дополнительно разделяется ещё на несколько очередей – по одной на каждый порт. Наконец, очереди в контроллере организованы в несколько уровней, и уровень очереди зависит от того, сколько коммутаторов или узлов она объединяет.

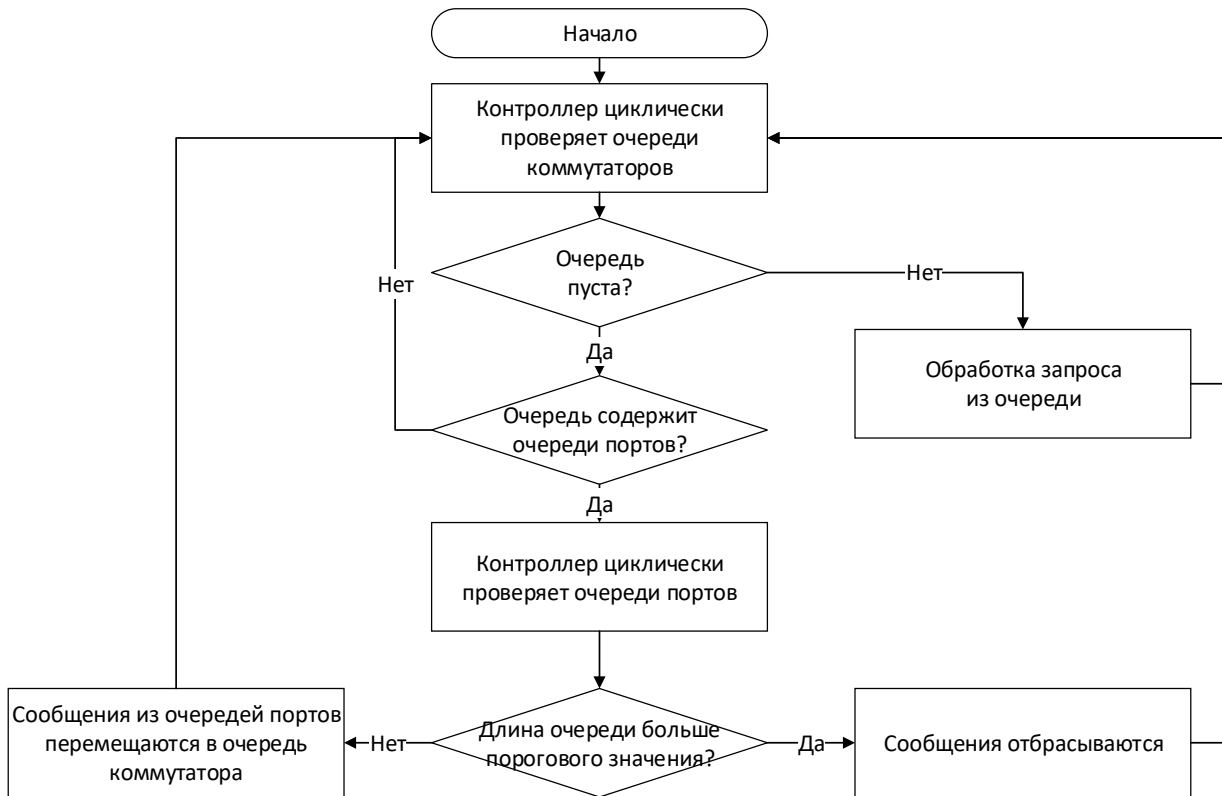


Рис. 2. Предложенный алгоритм обработки очередей на контроллере SDN

Длина очереди, используемая в экспоненциально взвешенное скользящее алгоритме (рис. 2), представляет собой среднее:

$$L_{avg} \leftarrow (1 - \alpha)_{avg} + \alpha L_{ins}, \quad (1)$$

где L_{avg} и L_{ins} – средняя и мгновенная длины очереди, а $0 < \alpha < 1$ – постоянная величина. должна быть слишком большой, чтобы предотвратить лишние перестроения очередей во время всплесков трафика в сети.

$$L + 1 + \frac{(1 - \alpha)^{L+1} - 1}{\alpha} < Th_{high}, \quad (2)$$

где L – число допустимых всплесков трафика.

Экспериментальная оценка

Для оценки эффективности предложенного механизма проведены экспериментальные испытания алгоритма путем программной модификации контроллера Floodlight [8]. Тестирование проведено посредством эмулятора компьютерных сетей Mininet v.2.2.2 [1, 9]

Ход эксперимента:

При помощи утилиты hping3 [10] на одном из хостов генерируются UDP-пакеты с

поддельными случайными адресами источника, каждый из которых вынуждает коммутатор-жертву отправить запроса потока на контроллер (рис. 3).

В то же время все остальные хосты в сети посредством утилиты ping проверяют связность друг с другом, чтобы оценить доступность контроллера.

Чтобы убедиться, что каждый пинг запускает запрос потока к контроллеру, время простоя установлено на 1 с, а интервал

запросов на 2 с. В рамках эксперимента размер очереди на контроллере выбран равным 1000, $\alpha = 0,01$, $Th_{high} = 10$, $Th_{low} = 2$.

Тогда, в соответствии с формулой (1), максимальное количество всплесков трафика в сети равно 48.

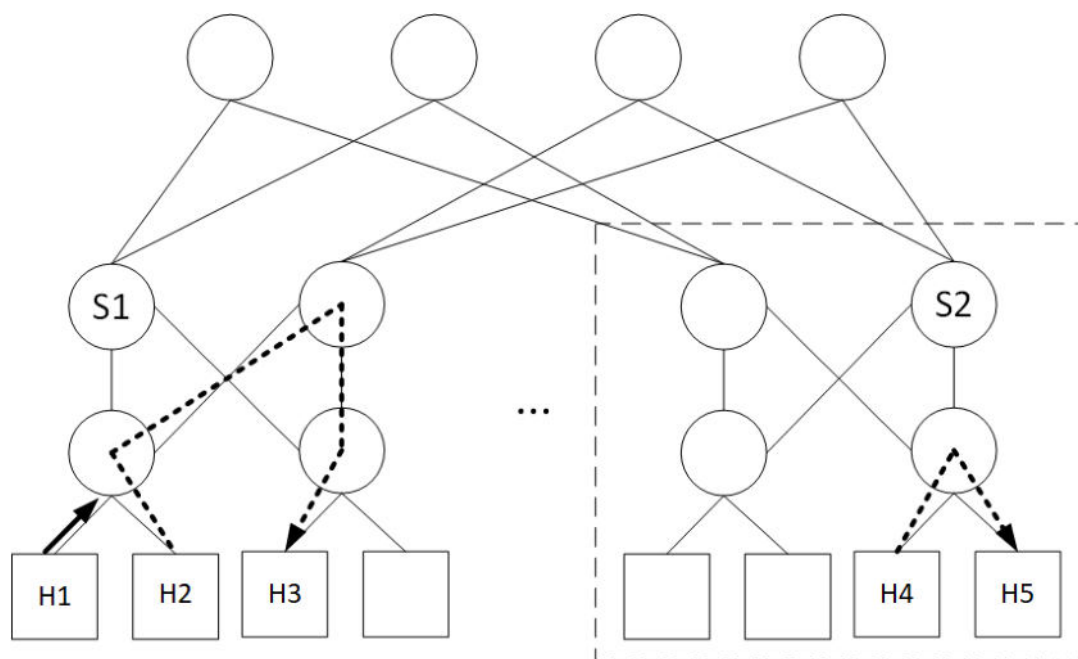


Рис. 3. Экспериментальная топология сети

Для проведения эксперимента смоделирована топология, состоящая из 20 коммутаторов Open vSwitch (OVS [11]) и 16 хостов.

В представленной схеме Н1 является атакующим устройством. Хост Н2 пингует Н3, а Н4 пингует Н5.

На рис. 4а показано время приема-передачи (RTT) между Н4 и Н5. Из него видно, что при использовании немодифицированного контроллера Floodlight значения RTT становятся чрезвычайно большими, когда количество запросов превышает 1000 в секунду. В то же время при использовании многоуровневой справедливой очереди RTT почти не

изменяется. Это демонстрирует эффективность предложенного механизма для обеспечения равноправной обработки пакетов разными коммутаторами.

На рис. 4б показано время приема-передачи между Н2 и Н3. Из него видно, многоуровневая очередь по-прежнему позволяет обеспечивать приемлемые значения RTT посредством разбиения очередей коммутатора на очереди портов. Это демонстрирует, что предложенный механизм может эффективно гарантировать равноправную обработку пакетов несколькими портами одного и того же коммутатора.

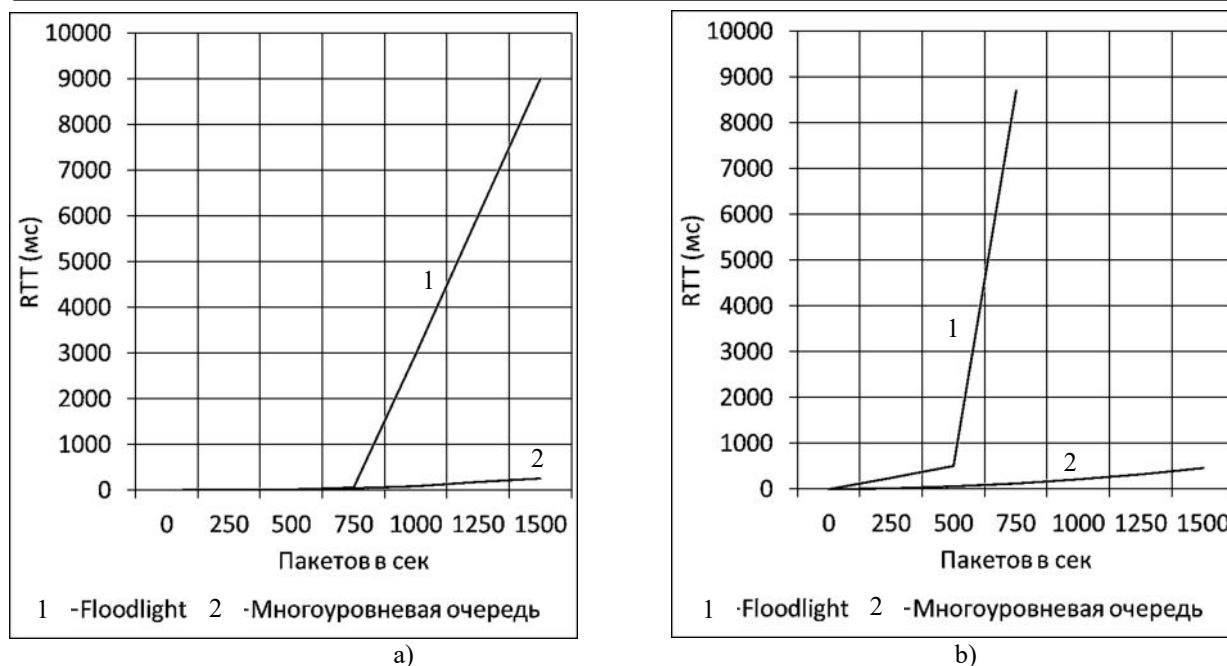


Рис. 4. Время приема-передачи между узлами H4 и H5; H2 и H3

Заключение

Предложенный механизм может решить проблемы поддержания слишком большого количества очередей путем их динамического расширения и агрегации в соответствии с нагрузкой контроллера. Многоуровневая справедливая очередь позволяет поддерживать небольшое количество очередей в нормальных ситуациях, а также эффективно изолировать вредоносные запросы в случае атаки отказа в обслуживании. Эксперименты, основанные на эмуляции, демонстрируют эффективность предложенного механизма.

Список литературы

1. Коннов А.Л. Анализ и проектирование программно-конфигурируемых сетей: учебное пособие / А.Л. Коннов, Ю.А. Ушаков, П.Н. Полежаев, В.В. Тугов. Оренбург: ОГУ, 2016. 114 с.
2. Руководство пользователя OpenFlow на оборудовании Juniper, URL: <https://www.juniper.net/documentation/us/en/software/junos/sdn-openflow/index.html> (дата обращения: 30.03.2021).
3. Singh, J. Detection and mitigation of DDoS attacks in SDN: A comprehensive review, research challenges and future directions / J. Singh, S. Behal // Computer Science Review 37. – 2020

4. Мухизи, С. Разработка моделей и методов сегментации ресурсов в программно-конфигурируемых сетях / С. Мухизи // Автореф. дис. канд. техн. Наук / СПбГУТ. Санкт-Петербург, 2019.

5. Спецификация коммутаторов OpenFlow, URL: <https://opennetworking.org/wp-content/uploads/2014/10/openflow-switch-v1.5.1.pdf> (дата обращения: 30.03.2021).

6. Nguyen-Ngoc A. Estimating the Flow Rule Installation Time of SDN Switches When Facing Control Plane Delay. / A. Nguyen-Ngoc, S. Lange, S. Geissler, T. Zinner, P. Tran-Gia // Measurement, Modelling and Evaluation of Computing Systems. 2018. P.113-126.

7. Apiecionek, L. Quality of Services Method as a DDoS Protection Tool [текст] / L. Apiecionek, J.M. Czerniak, W.T. Dobrosielski // Human-centric Computing and Information Sciences volume 9. 2019. P.225-230

8. Floodlight Project, URL: www.projectfloodlight.org/floodlight (дата обращения: 30.03.2021).

9. Эмулятор компьютерной сети Mininet, URL: www.mininet.org (дата обращения: 30.03.2021).

10. Сетевой анализатор Hping3, URL: www.hping.org (дата обращения: 30.03.2021).

11. Open vSwitch, URL: www.openvswitch.org (дата обращения: 30.03.2021).

Брянский государственный технический университет
Bryansk State Technical University

Поступила в редакцию 20.04.2021

Информация об авторах

Рытов Михаил Юрьевич – канд. техн. наук, доцент, Брянский государственный технический университет,
e-mail: rmy@tu-bryansk.ru

Калашников Руслан Юрьевич – аспирант, Брянский государственный технический университет,
e-mail: kalashnikov.r@protonmail.com

Горелов Алексей Алексеевич – аспирант, Брянский государственный технический университет,
e-mail: goralgod@yandex.ru

**APPLICATION OF MULTI-LEVEL FAIR QUEUE MECHANISM FOR MITIGATING
DENIAL-OF-SERVICE ATTACKS ON SOFTWARE-DEFINED NETWORKS**

M.Y. Rytov, R.Y. Kalashnikov, A.A. Gorelov

The concept of Software Defined Networking (SDN) is rapidly gaining popularity in the management of the network infrastructure of data centers and telecom operators. Its key functions include monitoring, granular control, flexibility and scalability. But at the same time, the centralized management of SDN makes it vulnerable to various types of attacks, such as spoofing and denial of service (DoS). DoS attacks have the most serious impact because they degrade network performance by overloading various components such as the controller, switch, and control channel. Existing approaches deal with SDN DoS attacks either by dropping malicious packets or by combining flow rules, which leads to the loss of legitimate traffic. To mitigate the impact of DoS attacks, this article proposes the use of a tiered fair queuing mechanism, which allows the sharing of controller resources with multiple queue tiers that can dynamically expand and aggregate based on network congestion. The proposed approach is evaluated by comparing it to a basic SDN controller. Simulation results show that the proposed approach increases SDN performance in terms of control channel bandwidth utilization.

Keywords: network security, denial of service, software defined networks.

Submitted 20.04.2021

Information about the authors

Mikhail Y. Rytov – Cand. Sc. (Technical), Associate Professor, Bryansk State Technical University,
e-mail: rmy@tu-bryansk.ru

Ruslan Y. Kalashnikov – graduate student, Bryansk State Technical University, e-mail: kalashnikov.r@protonmail.com

Alexey A. Gorelov – graduate student, Bryansk State Technical University, e-mail: goralgod@yandex.ru